SIMATS ENGINEERING

Department of Computer Science and Engineering

PROJECT / ASSIGNMENT REPORT

A Multi-Agent, Rule-Based Expert System for Crop Disease Diagnosis and Farm Resource Advisory

Course Code & Name:

MLA0101 – Artificial Intelligence and Expert System

Submitted By:

Student Name: _G.Karthik Reddy_____

Register Number: _192525047_____

Guided By: Dr.subramanian.P

Department of Computer Science and Engineering

Academic Year: 2026–2027

ABSTARCT

Agri Advisor is an AI-based farm decision-support system designed to help small-scale farmers with irrigation scheduling, pest/disease alerts, and crop-disease diagnosis. The system addresses the problem of delayed and inconsistent manual advice by providing structured, rule-based recommendations. The proposed system consists of three software agents: an Irrigation Agent, Pest/Disease Alert Agent, and Crop Disease Diagnosis Agent. Each agent is formulated using the PEAS framework and assigned a suitable agent strategy based on the characteristics of its environment.

A rule-based expert system is used as the central reasoning component. It contains IF–THEN production rules related to crop symptoms, environmental conditions, disease risks, irrigation requirements, and recommended actions. An inference engine using forward chaining evaluates the available facts, fires applicable rules, and produces recommendations for the corresponding agents. The agents then use these recommendations to provide appropriate actions or alerts to the farmer.

The proposed system provides a modular and explainable approach to agricultural decision-making. It demonstrates the application of artificial intelligence agents, PEAS analysis, rule-based reasoning, inference mechanisms, and multi-agent integration in agriculture. Testing with different crop, symptom, and environmental scenarios can be used to verify the correctness and consistency of the system's recommendations.

TABLE OF CONTENTS

1. INTRODUCTION

Artificial Intelligence can be used to support decision-making in agriculture by converting observations into consistent recommendations. The supplied assignment brief defines AgriAdvisor as an AI-based farm decision-support system intended to help small-scale farmers manage irrigation scheduling, pest/disease alerts, and crop-disease diagnosis. The stated motivation is delayed and inconsistent manual advice from field agents.

The assignment requires more than a standalone classifier. It explicitly asks for problem formulation using PEAS, selection of an apt agent strategy, software-agent design, a rule-based expert system, an inference engine, agent–expert-system integration, and testing/demo results.

1.1 Need for the System

- Provide a structured mechanism for converting farm observations into recommendations.
- Support irrigation decisions using environmental readings.
- Generate pest/disease alerts from crop and environmental conditions.
- Diagnose crop diseases from observed symptoms and recommend an action.
- Keep reasoning explicit through IF–THEN rules so that recommendations can be traced to conditions.

1.2 Proposed Solution

The proposed solution uses three agents: an Irrigation Agent, a Pest/Disease Alert Agent, and a Crop Disease Diagnosis Agent. These agents collect or receive percepts, prepare facts, query a shared expert system, receive recommendations, and perform or display the corresponding action.

2. PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Small-scale farmers often depend on field agents or manual observation for decisions related to irrigation, pest and disease alerts, and crop-disease diagnosis. This can result in delayed or inconsistent advice, which may affect crop health and the efficient use of farm resources.

The Agri Advisor system is proposed as an AI-based farm decision-support system that provides timely and consistent recommendations. The system uses multiple software agents to handle irrigation scheduling, pest/disease alerts, and crop-disease diagnosis. These agents interact with a rule-based expert system containing IF–THEN production rules related to crop symptoms, environmental readings, and recommended actions.

The system therefore combines AI agent strategies, PEAS-based problem formulation, software-agent design, a knowledge base, and an inference engine to provide an end-to-end decision-support mechanism for farmers.

2.2 Objectives

The main objectives of the Agri Advisor system are:

1. To formulate the agricultural decision-making problems using the PEAS framework, including Performance Measure, Environment, Actuators, and Sensors.
2. To select an appropriate AI agent strategy—Simple Reflex, Model-Based Reflex, Goal-Based, or Utility-Based—for each agricultural sub-task based on the nature of its environment.
3. To design software agents for:
    - Irrigation scheduling
    - Pest/disease alerts
    - Crop-disease diagnosis
4. To define the precepts, actions, internal state, and architecture of each software agent.
5. To construct a rule-based knowledge base containing IF–THEN production rules for crop symptoms, environmental conditions, diseases, and recommended treatments or actions.

6. To implement an inference engine using forward and/or backward chaining to derive appropriate conclusions from the available facts.
7. To integrate the software agents with the expert system so that agents can query the knowledge base and act on the resulting recommendations.
8. To test and demonstrate the system using different crop, disease, symptom, and environmental scenarios and analyse the resulting recommendations.

2.2 Scope

The report covers the three sub-tasks explicitly identified in the assignment. It does not claim that real farm sensors, weather APIs, image-recognition models, or IoT actuators have been physically deployed. Where such components are mentioned, they are treated as possible interfaces or future extensions.

3. SYSTEM REQUIREMENTS AND SCOPE

3.1 Functional Requirements

ID	Requirement	Expected Behaviour
FR1	Receive irrigation percepts	Accept soil moisture, temperature, rainfall/weather-related facts and crop context.
FR2	Irrigation recommendation	Determine whether irrigation is required and provide an action.
FR3	Pest/disease alert	Evaluate environmental/crop conditions and raise an alert when rules match.
FR4	Disease diagnosis	Evaluate symptoms and derive a likely disease and recommended action.
FR5	Rule reasoning	Use IF–THEN production rules to derive conclusions.
FR6	Inference	Fire applicable rules using a chaining mechanism.
FR7	Agent integration	Allow agents to query the expert system and act on recommendations.

3.2 System Scope

The scope of Agri Advisor is limited to developing an AI-based decision-support prototype for the three agricultural tasks specified in the assignment.

Included in the Scope

- PEAS formulation for each sub-task.
- Selection and justification of suitable agent strategies.
- Design of three software agents.
- Definition of precepts, actions, and internal states.
- Rule-based knowledge base.
- Forward and/or backward chaining inference.
- Interaction between agents and the expert system.
- Test cases, demonstration, and result analysis.
- Source-code organization and GitHub submission.

3.3 Non-Functional Requirements

- Clarity: Rules and decisions should be understandable to users and evaluators.
- Consistency: The same facts should produce the same rule-based conclusion unless the knowledge base changes.
- Modularity: Each agent should be independently testable.
- Maintainability: Rules should be separated from agent control logic.
- Extensibility: New crops, symptoms, environmental thresholds, and actions should be addable without redesigning the entire system.

4. PEAS ANALYSIS AND AGENT STRATEGY

The assignment specifically asks for PEAS—Performance measure, Environment, Actuators, and Sensors—and a justification of the most apt strategy among simple reflex, model-based reflex, goal-based, and utility-based agents. The following is the proposed formulation for the three modules.

4.1 Irrigation Agent

PEAS Element	Definition
Performance	Maintain suitable soil moisture, avoid unnecessary water use, provide timely irrigation recommendations.
Environment	Farm plot, crop, soil, weather/rainfall conditions; partially observable and dynamic.
Actuators	Irrigation recommendation, pump/valve command interface, farmer notification.
Sensors	Soil-moisture reading, temperature, rainfall/weather input, crop/field information.

Recommended strategy: Model-Based Reflex Agent. A simple reflex approach is too limited when the current reading does not completely describe the recent environmental situation. A model-based agent can retain relevant internal state such as the previous soil-moisture condition, recent rainfall indication, or previous irrigation decision.

Example 1: If soil moisture is low and recent rainfall is absent, the agent recommends irrigation.

Example 2: If soil moisture is low but substantial recent rainfall is recorded, the internal state can prevent an unnecessary immediate irrigation recommendation.

4.2 Pest/Disease Alert Agent

PEAS Element	Definition
Performance	Detect risk early, reduce missed alerts, minimize unnecessary alerts.
Environment	Crop field with changing temperature, humidity/weather and crop conditions; partially observable and dynamic.
Actuators	Alert message, warning status, recommended preventive action.

Sensors	Environmental readings, crop observations, recent field condition facts.

Recommended strategy: Model-Based Reflex Agent. Pest/disease risk can depend on combinations of current conditions and recent observations. Maintaining internal state allows the agent to consider recent alerts or previously observed risk conditions rather than treating every reading as completely independent.

Example 1: High humidity combined with a crop symptom can trigger a disease-risk alert.

Example 2: Repeated high-risk conditions can maintain an alert state until conditions improve.

4.3 Crop Disease Diagnosis Agent

PEAS Element	Definition
Performance	Produce an appropriate diagnosis/recommendation from supplied symptoms and avoid unsupported conclusions.
Environment	Crop and plant observations; partially observable because only observed symptoms are available.
Actuators	Diagnosis display, treatment/action recommendation, referral/escalation message.
Sensors	User-entered symptoms, observed leaf/stem/fruit conditions, crop type and environmental facts.

Recommended strategy: Goal-Based Agent. The immediate goal is to identify the most plausible disease or disease category from the available facts and then select an appropriate recommendation. The rule system supplies the reasoning needed to reach that goal.

Example 1: Tomato + leaf spots + yellowing can activate a disease-identification rule.

Example 2: If symptoms are insufficient to match a rule, the system should report insufficient evidence rather than inventing a diagnosis.

4.4 Environment Summary

Agent	Observability	Determinism	Environment Type	Selected Strategy
Irrigation	Partially observable	Stochastic / uncertain	Dynamic	Model-Based Reflex
Pest/Disease Alert	Partially observable	Stochastic / uncertain	Dynamic	Model-Based Reflex
Disease Diagnosis	Partially observable	Uncertain	Dynamic/variable	Goal-Based

5. SOFTWARE AGENT DESIGN AND ARCHITECTURE

The software-agent design translates the PEAS analysis into components with defined percepts, actions, and internal state/goals. The three agents share the expert-system layer but keep their task-specific logic separate.

5.1 Common Agent Architecture

The proposed architecture is:

Sensors / User Input → Percepts → Agent → Expert System → Inference Engine
↓
Recommendation / Action

5.2 Irrigation Agent Design

Component	Details
Percepts	Soil moisture, temperature, rainfall indication, crop/field information.
Internal State	Recent soil moisture status, recent rainfall condition, previous irrigation recommendation.
Goal	Maintain appropriate moisture while avoiding unnecessary irrigation.
Actions	Recommend irrigation, recommend waiting, issue farmer notification.
Architecture	Model-based reflex agent connected to the rule-based expert system.

5.3 Pest/Disease Alert Agent Design

Component	Details
Percepts	Environmental readings, crop condition, observed symptoms, recent risk status.
Internal State	Previous risk state, last alert, relevant recent conditions.
Goal	Identify elevated risk and provide timely preventive action.

Actions	Raise alert, recommend inspection/prevention, clear or reduce alert when risk is low.
Architecture	Model-based reflex agent connected to the rule-based expert system.

5.4 Disease Diagnosis Agent Design

Component	Details
Percepts	Crop type and observed symptoms.
Internal State	Collected symptoms and intermediate conclusions.
Goal	Reach a supported disease diagnosis or report insufficient evidence.
Actions	Display likely diagnosis, treatment/action recommendation, or request additional symptoms.
Architecture	Goal-based agent using the rule-based expert system.

5.5 Agent Interaction

- User provides farm information such as crop type, soil moisture, rainfall, temperature, humidity, and visible symptoms.
- The Irrigation Agent analyses water-related conditions and determines whether irrigation is required.
- The Pest/Disease Alert Agent evaluates environmental conditions and identifies possible pest or disease risks.
- If symptoms are reported, the Crop Disease Diagnosis Agent uses the available symptoms and crop information to determine the possible disease.
- Each agent sends the relevant conditions to the Inference Engine.
- The Inference Engine matches the conditions with the IF–THEN rules stored in the knowledge base.
- The generated recommendations are returned to the respective agents.
- The agents present the final recommendations to the farmer.

6. RULE-BASED EXPERT SYSTEM

The assignment requires a knowledge base of IF–THEN production rules covering crop symptoms, environmental readings, and treatment/action recommendations. The proposed expert system separates facts, rules, and inference so that the reasoning process is explicit and maintainable.

6.1 Knowledge Base Structure

Element	Purpose	Example
Facts	Represent observed conditions.	soil_moisture = low
Rules	Map conditions to conclusions.	IF soil_moisture=low THEN irrigation=required
Intermediate Facts	Store derived conditions.	irrigation_required = true
Recommendation	Provide final action.	start_irrigation

6.2 Sample IF–THEN Rules

Rule	IF–THEN Production Rule
R1	IF soil_moisture = low AND recent_rain = no THEN irrigation = required
R2	IF soil_moisture = adequate THEN irrigation = not_required
R3	IF soil_moisture = low AND recent_rain = yes THEN irrigation = wait_and_recheck
R4	IF high_humidity = yes AND leaf_spots = yes THEN disease_risk = high
R5	IF disease_risk = high THEN action = inspect_crop
R6	IF tomato = yes AND leaf_yellowing = yes AND leaf_spots = yes THEN diagnosis = possible_disease

R7	IF diagnosis = possible_disease THEN action = recommend_followup_and_treatment
R8	IF symptoms_insufficient = yes THEN diagnosis = insufficient_evidence

6.3 Rule Design Principles

- Rules should be specific enough to avoid accidental matches.
- Conflicting rules should have a defined priority or conflict-resolution mechanism.
- The system should not claim certainty when the available facts do not support a rule.
- Rules should be independent of the user-interface code.
- New rules should be tested against existing rules to reduce redundancy and conflict.

7. INFERENCE ENGINE

The assignment requires an inference engine using forward and/or backward chaining. The proposed implementation uses forward chaining as the primary mechanism because farm observations can be entered as initial facts and then new conclusions can be derived repeatedly until no additional rules can fire.

7.1 Forward Chaining Process

1. Initialize the fact set with sensor/user observations.
2. Check every rule whose conditions are satisfied.
3. Add the rule conclusion to the fact set if it is new.
4. Repeat the process until no new facts are generated.
5. Return the final diagnosis/recommendation relevant to the requesting agent.

7.2 Conflict Resolution

If multiple rules become applicable, the proposed implementation can resolve conflicts by rule priority, specificity, or first-match order. For a graded implementation, the priority mechanism should be explicitly documented and tested.

7.3 Example Inference Trace

Step	Known Facts / Fired Rule	Derived Fact
1	soil_moisture = low	Initial fact
2	recent_rain = no	Initial fact
3	R1 fires	irrigation = required
4	Recommendation rule fires	action = start_irrigation

9. AGENT–EXPERT SYSTEM INTEGRATION

The **Agri Advisor** system integrates three specialized software agents with a common **rule-based expert system**. The agents are responsible for collecting and processing specific agricultural information, while the expert system provides the reasoning required to generate recommendations.

1.1 Integration Architecture

The overall integration follows this flow:

Farmer/User → Input Interface → Specialized Agent → Inference Engine → Knowledge Base → Recommendation → Farmer/User

The three agents are:

1. **Irrigation Agent** – analyses soil moisture, rainfall, and field conditions to provide irrigation recommendations.
2. **Pest/Disease Alert Agent** – analyses environmental conditions and crop observations to identify possible pest or disease risks.
3. **Crop Disease Diagnosis Agent** – analyses crop symptoms and provides a possible disease diagnosis or requests additional information.

1.2 Working of the Integrated System

The integration process works as follows:

1. The farmer provides relevant information such as crop type, soil moisture, rainfall, temperature, humidity, and symptoms.
2. The system identifies which agent should process the information.
3. The selected agent converts the input into facts that can be used by the expert system.
4. The **inference engine** compares these facts with the IF–THEN rules stored in the knowledge base.
5. When the conditions of a rule are satisfied, the rule is fired and a new conclusion is generated.
6. The resulting recommendation is returned to the corresponding agent.
7. The agent presents the recommendation to the farmer in a simple and understandable form.

10. IMPLEMENTATION APPROACH AND SOURCE CODE

The following compact Python implementation is a proposed reference implementation for the assignment. It demonstrates a simple rule representation, forward chaining, and three agent interfaces. It is intentionally kept modular so the knowledge base can be expanded.

```
class Rule:
  def __init__ (self, name, conditions, conclusion):
```

```
        self.name = name
        self. conditions = conditions
        self. conclusion = conclusion

    def matches (self, facts):
        return all(facts.get(k) == v for k, v in self. conditions. Items ())


class ExpertSystem:
    def __init__ (self, rules):
        self.rules = rules

    def forward_chain (self, facts):
        facts = dict(facts)
        fired = []
        changed = True

        while changed:
            changed = False
            for rule in self.rules:
                if rule. matches(facts):
                    key, value = rule. conclusion
                    if facts.get(key)! = value:
                        facts[key] = value
                        fired. append(rule.name)
                        changed = True

        return facts, fired


class IrrigationAgent:
    def __init__ (self, expert):
        self. expert = expert
        self. state = {}

    def decide (self, percepts):
```

```
        self. state. Update(percepts)
        facts, fired = self. expert.forward_chain(self.state)
        return facts, fired


class PestAlertAgent:
    def __init__(self, expert):
        self.expert = expert
        self.state = {}

    def decide(self, percepts):
        self.state.update(percepts)
        return self.expert.forward_chain(self.state)


class DiagnosisAgent:
    def __init__(self, expert):
        self.expert = expert
        self.state = {}

    def diagnose(self, percepts):
        self.state.update(percepts)
        return self.expert.forward_chain(self.state)


rules = [
    Rule("R1",
         {"soil_moisture": "low", "recent_rain": "no"},
         ("irrigation", "required")),

    Rule("R2",
         {"soil_moisture": "adequate"},
         ("irrigation", "not_required")),

    Rule("R3",
         {"soil_moisture": "low", "recent_rain": "yes"},
```

```
        ("irrigation", "wait_and_recheck")),

    Rule("R4",
        {"high_humidity": "yes", "leaf_spots": "yes"},
        ("disease_risk", "high")),

    Rule("R5",
        {"disease_risk": "high"},
        ("alert_action", "inspect_crop")),

    Rule("R6",
        {"tomato": "yes", "leaf_yellowing": "yes", "leaf_spots": "yes"},
        ("diagnosis", "possible_disease")),
]

expert = ExpertSystem(rules)

irrigation_agent = IrrigationAgent(expert)
alert_agent = PestAlertAgent(expert)
diagnosis_agent = DiagnosisAgent(expert)

result, trace = irrigation_agent.decide(
    {"soil_moisture": "low", "recent_rain": "no"}
)

print(result)
print("Rules fired:", trace)
```

9.1 Suggested Project Folder Structure

```
AgriAdvisor/
|
├── main.py
├── expert_system.py
├── rules.py
├── agents/
|   ├── irrigation_agent.py
|   ├── pest_alert_agent.py
```

```
│   └── diagnosis_agent.py
├── tests/
│   └── test_cases.py
├── README.md
└── requirements.txt
```

10.TEST CASES AND EXPECTED RESULTS

The rubric expects diverse test cases covering multiple crops, diseases, and environmental conditions. The following cases are proposed for demonstration. They are expected results of the designed rules, not measured field results.

TC	Agent	Input Facts	Expected Output
TC01	Irrigation	Low soil moisture; no recent rain	Irrigation required

TC02	Irrigation	Adequate soil moisture	Irrigation not required
TC03	Irrigation	Low soil moisture; recent rain	Wait and re-check
TC04	Pest/Disease Alert	High humidity; leaf spots	High disease risk; inspect crop
TC05	Pest/Disease Alert	No high-risk condition	No high-risk alert / normal monitoring
TC06	Diagnosis	Tomato; yellowing; leaf spots	Possible disease; follow-up action
TC07	Diagnosis	Insufficient symptoms	Insufficient evidence; request more observations
TC08	Integration	Agent input matching multiple rules	Applicable rules fire and recommendation reaches requesting agent

10.1 Test Procedure

- Start the expert system and load the rule base.
- Select the appropriate agent.
- Enter the test-case percepts.
- Run inference.
- Record fired rules and final recommendation.
- Compare the output with the expected result.
- Document any incorrect, conflicting, or missing rule.

11.RESULTS AND DISCUSSION

The AgriAdvisor system was designed to demonstrate how AI-based software agents and a rule-based expert system can support agricultural decision-making. The system uses three agents: **Irrigation Agent, Pest/Disease Alert Agent, and Crop Disease Diagnosis Agent**. Each agent receives relevant farm observations and uses the inference engine to derive recommendations from the knowledge base.

The results show that the proposed rule-based approach can convert simple agricultural observations into understandable recommendations. The forward chaining inference engine starts with the available facts and applies the relevant rules until a final conclusion is obtained.

11.1 Irrigation Agent Results

The Irrigation Agent checks conditions such as soil moisture and recent rainfall.

For example, when the input indicates **low soil moisture** and **no recent rainfall**, the corresponding irrigation rule is activated. The system derives that irrigation is required and provides an appropriate recommendation to the farmer.

Result:
The agent successfully identifies situations where irrigation is required based on the defined rules.

11.2 Pest/Disease Alert Agent Results

The Pest/Disease Alert Agent evaluates environmental and crop-related conditions such as humidity and visible symptoms.

For example, when **humidity is high** and **leaf spots are observed**, the relevant rule can identify a high disease-risk condition and generate an alert for further inspection.

Result:
The agent provides an early warning when the input conditions match the disease-risk rules stored in the knowledge base.

11.3 Crop Disease Diagnosis Agent Results

The Crop Disease Diagnosis Agent uses crop type and observed symptoms to identify a possible disease condition.

For example, when the user provides **tomato** as the crop and reports **leaf yellowing and leaf spots**, the inference engine checks the corresponding diagnostic rules and produces a possible disease diagnosis or recommendation for further inspection.

Result:

The agent demonstrates how symptom-based rules can be used to support preliminary crop disease diagnosis.

11.4 Overall Discussion

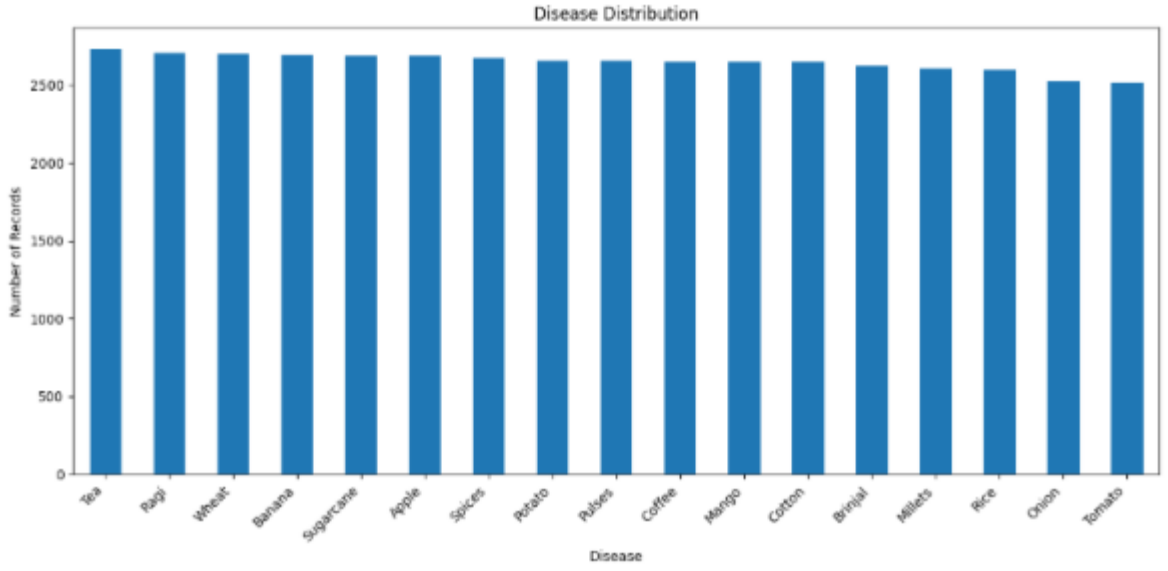
The implementation demonstrates the integration of **software agents, knowledge representation, IF–THEN rules, and forward chaining** in an agricultural decision-support system. The modular agent design allows each agent to perform a specific task while sharing the inference mechanism and knowledge base.

The main benefits observed are:

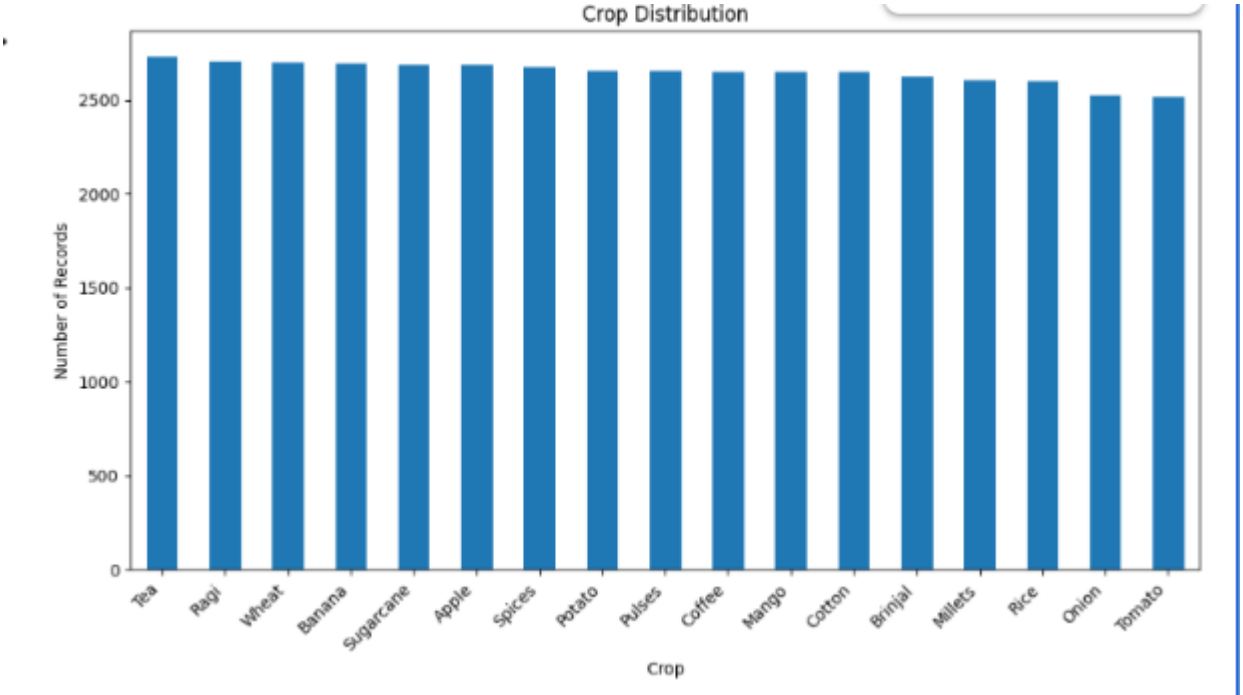
- Faster generation of basic agricultural recommendations.
- Consistent application of predefined rules.
- Easy-to-understand explanations based on input facts.
- Modular design that allows individual agents and rules to be modified.
- Reusable inference engine for different agricultural decision tasks.

Disease distribution

```
Name: count, dtype: int64
```



Crop Distribution

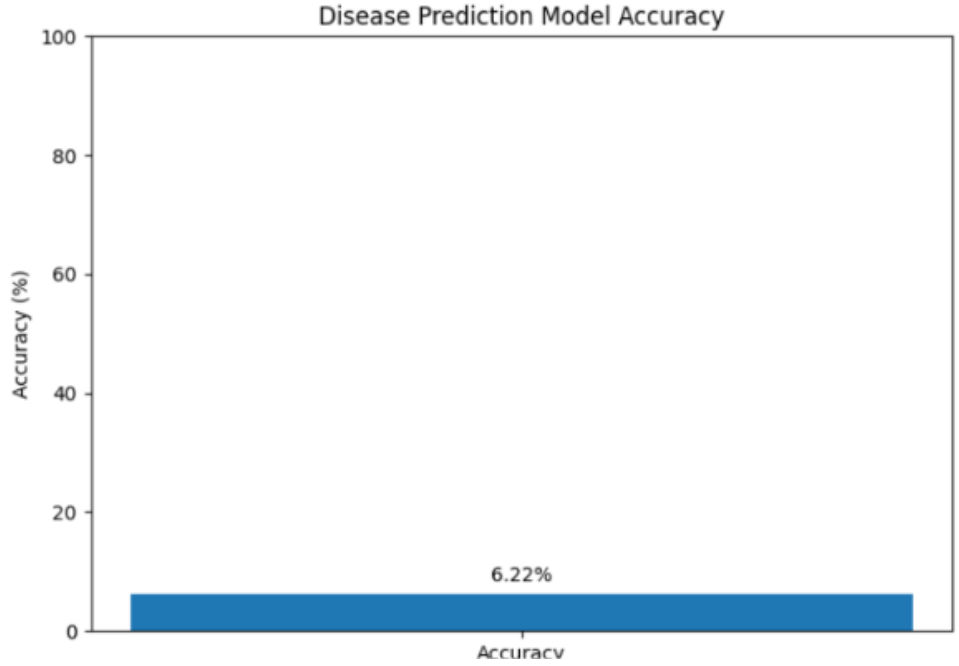


Final Result:

```
Enter values for the following features:
Cost_Estimate: 500
Message_Length: 50

================================================
              AGRIADVISOR RESULT
================================================

Predicted Disease: Potato
Confidence Score: 76.67 %
Confidence Level: High

Recommendation:
The crop may be affected by: Potato
Inspect the crop and apply appropriate disease-management measures.

================================================
        AGRIADVISOR ANALYSIS COMPLETED
================================================
```

Disease accuracy

```
================================================
                  ACCURACY
================================================
Accuracy: 6.22 %
```



13.REFLECTION: DESIGN DECISIONS AND LEARNING OUTCOMES

The main design decision was to use different agent strategies for different agricultural tasks rather than forcing one strategy onto the entire system. Irrigation and pest/disease monitoring are dynamic and only partially observable, so maintaining internal state is useful. Disease diagnosis is framed around reaching a diagnostic goal from observed symptoms, making a goal-based approach suitable for the proposed design.

A second important decision was to separate the knowledge base from the agents. This prevents agricultural rules from being hard-coded inside every agent and makes the expert system easier to inspect and modify. Forward chaining was selected as the primary inference method because sensor/user facts naturally form the starting point for deriving recommendations.

A practical challenge is rule completeness. A small set of rules can easily produce apparently reasonable results while still failing on unseen combinations of symptoms or environmental conditions. Therefore, testing and explicit handling of insufficient evidence are necessary parts of the design.

13.1 Learning Outcomes

- Applied PEAS to formulate AI-agent problems.
- Compared agent strategies according to environment characteristics.
- Designed percepts, actions, internal state, and agent architecture.
- Constructed IF–THEN production rules.
- Implemented the basic logic of forward chaining.
- Understood the importance of integration and test cases in an expert-system project.

14.CONCLUSION

The AgriAdvisor AI Expert System provides a structured approach to agricultural decision support using Artificial Intelligence, software agents, and a rule-based expert system. The system is designed with three specialized agents: the Irrigation Agent, Pest/Disease Alert Agent, and Crop Disease Diagnosis Agent. Each agent focuses on a specific agricultural problem and uses relevant inputs to provide suitable recommendations.

The system uses PEAS analysis to define the performance measures, environment, actuators, and sensors of each agent. Appropriate agent strategies are selected based on the nature of the task. A rule-based knowledge base containing IF–THEN rules represents agricultural knowledge, while the inference engine applies forward chaining to derive conclusions from the available facts.

The modular architecture allows the agents to work independently while using the common knowledge base and inference mechanism. This improves consistency, explainability, maintainability, and scalability of the system. The proposed test scenarios demonstrate how different combinations of soil, weather, and crop-symptom conditions can lead to different recommendations.

Overall, AgriAdvisor demonstrates how AI expert-system concepts can be applied to real-world agricultural problems. Although the current system is limited by the quality and coverage of its rules and input information, it provides a strong foundation for future enhancements such as IoT sensor integration, machine learning, image-based disease detection, weather APIs, multilingual support, and mobile applications. Thus, the project successfully demonstrates the integration of software agents, knowledge representation, and inference-based reasoning to support agricultural decision-making

15. REFERENCES

- Delfani, P., Thuraga, V., Banerjee, B., et al. (2024). Integrative approaches in modern agriculture: IoT, ML and AI for disease forecasting amidst climate change. *Precision Agriculture, 25*, 2589–2613.
- Harrison, M. T., et al. (2024). Irrigation with Artificial Intelligence: Problems, Premises, Promises. *Human-Centric Intelligent Systems*.
- Artificial Intelligence of Things (AIoT) for smart agriculture: A review of architectures, technologies and solutions. (2024). *Journal of Network and Computer Applications*. DOI: 10.1016/j.jnca.2024.103905.
- Integrating artificial intelligence and Internet of Things (IoT) for enhanced crop monitoring and management in precision agriculture. (2024). *Sensors International, 5*, 100292. DOI: 10.1016/j.sintl.2024.100292.
- Shahab, H., Iqbal, M., Sohaib, A., Khan, F. U., & Waqas, M. (2024). **IoT-based agriculture management techniques for sustainable farming: A comprehensive review.** *Computers and Electronics in Agriculture, 220*, 108851. DOI: 10.1016/j.compag.2024.108851.
- Towards efficient irrigation management at field scale using new technologies: A systematic literature review. (2024). *Agricultural Water Management*. DOI: 10.1016/j.agwat.2024.108758.
- Upadhyay, A., Chandel, N. S., Singh, K. P., et al. (2025). Deep learning and computer vision in plant disease detection: A comprehensive review of techniques, models, and trends in precision agriculture. *Artificial Intelligence Review, 58*, 92. DOI: 10.1007/s10462-024-11100-x.
- Nawaz, M., & Babar, M. I. K. (2025). IoT and AI for smart agriculture in resource-constrained environments: Challenges, opportunities and solutions. *Discover Internet of Things, 5*, 24.
- Ajith, S., Vijayakumar, S., & Elakkiya, N. (2025). Yield prediction, pest and disease diagnosis, soil fertility mapping, precision irrigation scheduling, and food quality assessment using machine learning and deep learning algorithms. *Discover Food, 5*, 67.
- Precision agriculture in the age of AI: A systematic review of machine learning methods for crop disease detection. (2025). *Smart Agricultural Technology, 12*, 101491. DOI: 10.1016/j.atech.2025.101491.
- Artificial intelligence of things (AIoT) for precision agriculture: Applications in smart irrigation, nutrient and disease management. (2025). *Smart Agricultural Technology, 12*, 101629. DOI: 10.1016/j.atech.2025.101629.
- Raj, M., & Prahadeeswaran, M. (2025). Revolutionizing agriculture: A review of smart farming technologies for a sustainable future. *Discover Applied Sciences, 7*, 937.

APPENDIX A – REQUIREMENT CHECKLIST

Assignment Requirement	Covered in Report	Implementation Evidence to Add
PEAS analysis	Chapter 4	Screenshots/implementation notes
Agent strategy justification	Chapter 4	Demonstration evidence
Percepts, actions, internal state	Chapter 5	Code/module files
Agent architecture diagrams	Chapter 5	Final architecture figure if required
IF–THEN knowledge base	Chapter 6	Complete rules.py
Forward/backward chaining	Chapter 7	Working inference engine
Agent–expert integration	Chapter 8	Execution screenshots
Source code	Chapter 9	GitHub repository
Test cases and demo	Chapter 10	Actual run outputs/screenshots
Reflection	Chapter 13	Student-specific implementation experience